ClauseLens - Frontend Design Spec (React)

Stack: React 18 + Vite + TypeScript + Tailwind CSS, `lucide-react`, fonts bundled with `@fontsource/*` **Design goal:** make the difference between what the document says and what it might mean visible at a glance, and make privacy visible without a lecture.

Visual polish earns little in the rubric. Clear grounding, honest uncertainty and a usable flow for a non-lawyer earn points. Build in that order.

> **Privacy rule for the frontend:** no CDN, no Google Fonts, no analytics, no remote images, no `localStorage` of document content. Every asset is bundled. The backend sends a strict CSP, so anything external will simply fail and show up as a bug.

---

1. Design principles

1. **Source first.** The document is always visible next to the analysis. Every claim links to a highlighted passage.
2. **Says vs. might mean.** Two clearly different visual treatments, never colour alone.
3. **Uncertainty is shown, not hidden.** Each finding carries a level and reasons.
4. **Silence is a result.** "The document does not say" gets its own place in the UI.
5. **Calm tone.** Plain words, no alarm colours for ordinary clauses. Reserve strong emphasis for items that need a professional.
6. **Always local, always not-advice.** A privacy badge and a disclaimer are permanent.

2. Design direction (for this brief)

The product is a reading aid for a signed-in-ink kind of document, so the interface behaves like an annotated page: a document column with margin-style annotations, not a chat window and not a card dashboard.

Axis	Choice
Base palette	Paper `#F4F5F2` (cool grey-white), surface `#FFFFFF`, ink `#16212B`, muted ink `#4A5A68`, rule `#D5DAD6`
Accents	"Marker" highlight `#CFE8A9` for explicit text; interpretation underline `#6B4FBB` (dashed); attention `#A15C00`; needs-professional `#A3262E`
Dark mode	Optional stretch; do not start it unless Block B is finished
Type	Document text and findings titles: a serif (`@fontsource/literata`) for the reading feel. UI chrome: a clean sans (`@fontsource/public-sans`). Both bundled. Fall back to `Georgia, serif` and `system-ui, sans-serif`
Scale	16px base, 1.6 line height for serif reading text, line length under 75 characters in the document pane
Radius	6px on controls, 0 on the document page (it is paper), 10px on finding cards
Motion	None except focus and tab changes; a single highlight pulse when a citation is clicked (disabled under `prefers-reduced-motion`)

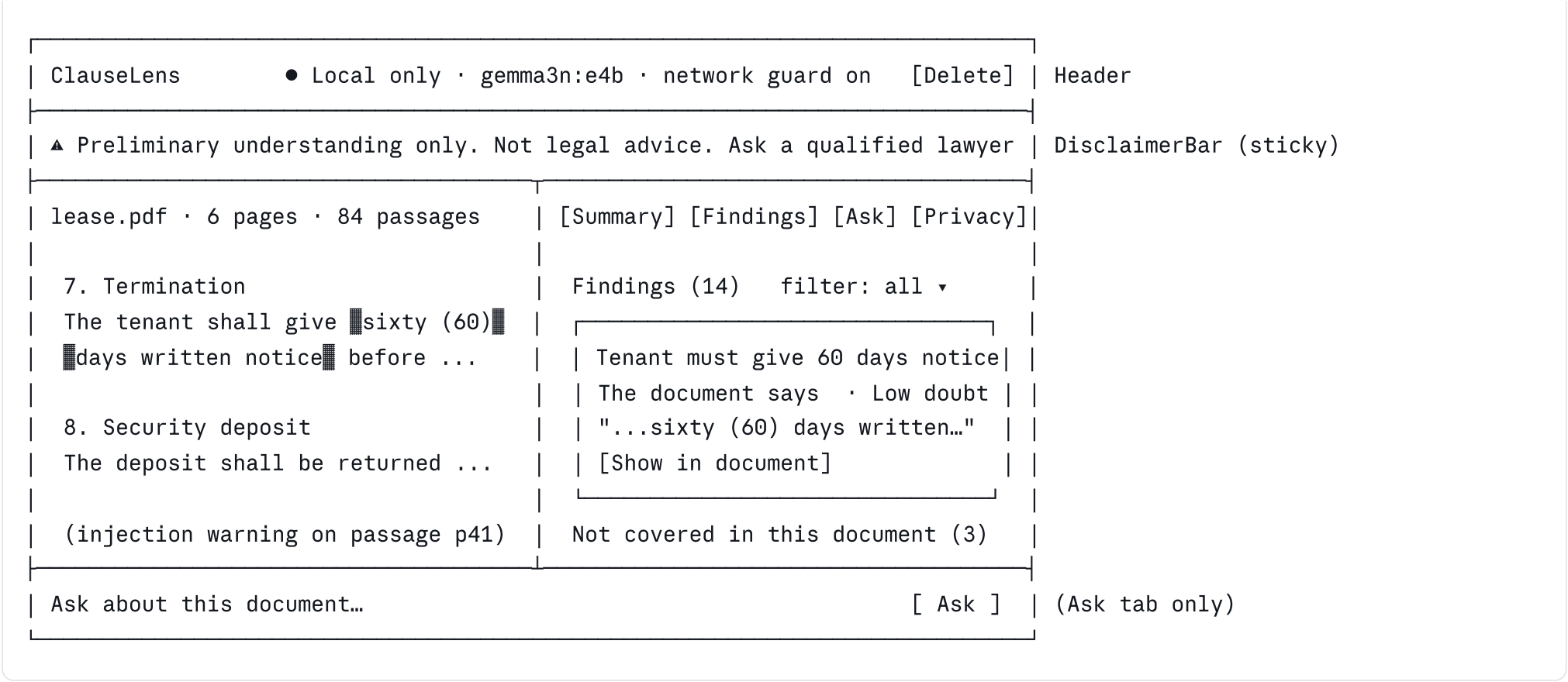
One memorable thing: the citation link. Clicking a finding scrolls the document to the passage and paints the quoted words with a marker highlight, with a thin connector label "Quote verified". Everything else stays quiet.

Contrast: all text pairs at 4.5:1 or better (check marker highlight on ink text; ink on `#CFE8A9` passes).

3. Layout

Desktop (two panes): document left (about 55%), analysis right (about 45%). Mobile: tabs switch between "Document" and "Analysis".

```text
```



Empty state (no document): a single large drop zone: "Open a document. It stays on this computer." with file picker for PDF or text, and a link "Try a sample rental agreement" (loads the bundled synthetic sample via the real backend).

4. Visual language for the key distinctions

Meaning	Treatment	Text label	Icon
The document says (`explicit`)	Solid marker highlight on the quoted words in the document; card has a solid left rule	"The document says"	quote
Possible interpretation (`interpretation`)	Dashed violet underline on the passage; card has a dashed left rule	"This may mean"	lightbulb
Concern (`concern`)	Dashed amber underline; card has a dashed left rule	"Worth checking"	alert-triangle
Not in document (`silent`)	Hollow outlined chip, no highlight	"The document does not say"	file-question
Needs a professional (`escalate`)	Red-brown tag on the card	"Ask a lawyer about this"	scale
Injection suspected	Hatched border on the passage	"This text contains instructions aimed at AI tools. They were ignored."	shield

Never rely on colour only: every state has a text label and an icon, and solid vs dashed lines differ in form.

Uncertainty meter (`ConfidenceMeter`): three-step bar with text "Low doubt / Some doubt / High doubt", plus the reasons as plain phrases: "Wording is vague", "Term is not defined", "Clauses conflict", "Depends on local law", "Text was hard to read".

5. Screens and states

State	What the user sees
Empty	Drop zone, sample link, one line on privacy
Uploading / reading	Progress text "Reading the document on this computer"
Upload error	Plain message and fix: "This PDF has no selectable text (it may be a scan). Scanned files are not supported yet. Try a text-based PDF or a .txt file."
Analysing	Stage list: Reading, Finding clauses, Checking quotes, Writing summary. Partial findings appear as they verify
Done	Summary, findings, silent list, escalation panel
Partial	Banner "Some sections could not be analysed" (from malformed model output)
Model unavailable	"The local model is not running. Start Ollama and press Retry." with Retry
Injection suspected	Warning chip in header of the document, hatched passage
Ask: answered	Answer text, citations with Show in document, kind and uncertainty
Ask: silent	"The document does not say." plus nearest related passages if any, and a suggestion to ask the other party or a lawyer
Ask: limited	"I can't say whether this is enforceable." plus what the text states, why, and a specific referral
Ask: off topic	One line redirect
Network guard blocked	Privacy tab shows a red event count if anything was blocked (should be zero)

6. Components

```text
src/
├── main.tsx
├── App.tsx
├── types.ts
├── api/client.ts
├── hooks/
│   ├── useDocument.ts
│   ├── useAnalysis.ts          # starts job, polls /api/jobs/:id
│   └── useAsk.ts
└── components/
    ├── Header.tsx
    ├── DisclaimerBar.tsx
    ├── UploadPanel.tsx
    ├── DocumentViewer.tsx
    ├── PassageBlock.tsx
    ├── SummaryTab.tsx
    ├── FindingsTab.tsx
    ├── FindingCard.tsx
    ├── CitationChip.tsx
    ├── ConfidenceMeter.tsx
    ├── SilentList.tsx
    ├── EscalationPanel.tsx
    ├── AskTab.tsx
    ├── PrivacyTab.tsx
    └── StageProgress.tsx
```

Component	Props	Notes
`App`	none	Holds `doc`, `analysis`, `activeCitation`; two-pane layout; tab state
`Header`	`health`, `onDelete`	"Local only" badge from `/api/health` (`offline_mode`, `network_guard`); model name
`DisclaimerBar`	none	Sticky, not dismissible. Text: "Preliminary understanding only. Not legal advice. For decisions, ask a qualified legal professional."
`UploadPanel`	`onFile`, `onSample`, `busy`	Native file input plus drag and drop; accepts `.pdf,.txt`
`DocumentViewer`	`passages`, `highlights`, `active`, `injectionFlags`	Renders passages in order, serif text, page markers; scrolls to `active`; paints highlights from verified quotes
`PassageBlock`	`passage`, `ranges`, `kind`, `flagged`	Splits text into plain and highlighted spans using quote match offsets
`SummaryTab`	`analysis`	Summary, document type, parties, key dates table, escalation panel
`FindingsTab`	`findings`, `onCite`	Filter by category and kind; sorted by severity
`FindingCard`	`finding`, `onCite`	Label (says / may mean / worth checking), title, plain explanation, quote(s), uncertainty, escalate tag
`CitationChip`	`citation`, `onClick`	Shows page and short quote; button "Show in document"
`ConfidenceMeter`	`uncertainty`	Text plus bar; reasons as phrases
`SilentList`	`items`	"Not covered in this document" with a note each
`EscalationPanel`	`text`	Specific referral text from the backend

Component	Props	Notes
`AskTab`	`docId`, `onCite`	Input with label, message list (`aria-live="polite"`), response renderer by status
`PrivacyTab`	`privacy`, `health`	Declared network calls table, guard status, how to verify offline (link to docs text shown inline)
`StageProgress`	`stage`, `progress`	Reading, Finding clauses, Checking quotes, Writing summary

Highlighting algorithm

The backend returns `passages[]` with `text` and each citation's `quote`. In `PassageBlock`, find the quote inside the passage with the same normalisation the backend uses (collapse whitespace, normalise quotes and dashes, case-insensitive). Map back to original character offsets and split into spans. If the quote is not found (should not happen for verified citations), highlight the whole passage and show a small "approximate match" note. Do not invent highlights.

7. Interaction flows

- **Open a document:** pick file, upload, passages appear immediately in the viewer, analysis starts automatically (or on a button press if you prefer control), progress shows.
- **Cite:** click "Show in document" on a finding or answer. Viewer scrolls, passage gets focus and the highlight pulse. Keyboard: citation button is focusable; pressing it moves focus to the passage (`tabIndex={-1}` plus `focus()`).
- **Ask:** type question, Enter to send, Shift+Enter newline. Disabled while loading. Answer renders by `status`.
- **Delete:** confirm dialog "This removes the document and results from this computer." Then `DELETE /api/documents/:id` and reset state.
- **Retry on model unavailable:** button re-calls analyze or ask.
- **Sample document:** loads the synthetic sample through the real upload endpoint, never a canned result (gate G3).

8. Copy rules

- Say "The document says" for explicit text. Say "This may mean" or "Worth checking" for the rest.
- Never write "legal", "illegal", "valid", "void", "enforceable" as a conclusion. Those words may only appear inside quoted document text or inside "I can't say whether this is enforceable".
- Escalation wording is specific: "Ask a tenant or employment lawyer to review clause 7 (early termination)."
- Errors state what happened and what to do next. No apologies, no blame.
- Sentence case everywhere.

9. Accessibility checklist

- [ ] Everything operable by keyboard; visible focus ring (2px, offset)
- [ ] Inputs have labels; icon-only buttons have `aria-label`
- [ ] Tabs follow the ARIA tab pattern with arrow keys
- [ ] New answers and progress announced via `aria-live="polite"`
- [ ] Highlights have a text equivalent (finding label) and do not rely on colour alone (solid vs dashed, icons, labels)
- [ ] Contrast 4.5:1 or better; check marker highlight and muted text
- [ ] Document pane is a labelled `region`; passages have `id` anchors
- [ ] Respect `prefers-reduced-motion`
- [ ] Works at 360px width; layouts collapse to tabs
- [ ] Reading order matches visual order

10. Types (`src/types.ts`)

```ts
```

```ts
export type Kind = "explicit" | "interpretation" | "concern";
export type Severity = "info" | "attention" | "high";
export type Category =
  | "obligation" | "right" | "deadline" | "penalty" | "termination"
  | "payment" | "renewal" | "liability" | "dispute" | "other";
export type UncertaintyReason =
  | "vague_wording" | "undefined_term" | "conflicting_clauses"
  | "jurisdiction_dependent" | "incomplete_text" | "poor_extraction";

export interface Passage {
  id: string; page: number; start: number; end: number;
  heading: string | null; text: string;
}
export interface Citation { passage_id: string; quote: string; verified: boolean; }
export interface Uncertainty {
  level: "low" | "medium" | "high"; reasons: UncertaintyReason[]; note: string | null;
}
export interface Finding {
  id: string; category: Category; title: string; plain_explanation: string;
  kind: Kind; severity: Severity; citations: Citation[];
  uncertainty: Uncertainty; escalate: boolean;
}
export interface SilentItem { topic: string; note: string; }
export interface KeyDate { label: string; value: string; citation?: Citation; }
export interface Analysis {
  document_type: "rental" | "employment" | "other" | "unknown";
  summary: string; parties: string[]; key_dates: KeyDate[];
  findings: Finding[]; silent: SilentItem[]; escalation: string;
  disclaimer: string; injection_flags: string[]; dropped_unverified: number;
  timings_ms: Record<string, number>; model_info: Record<string, string>;
```

```ts
}
export interface UploadResult {
  doc_id: string; filename: string; pages: number; passage_count: number;
  char_count: number; warnings: string[]; passages: Passage[];
}
export interface Job {
  state: "running" | "done" | "error";
  stage: "reading" | "finding" | "verifying" | "summarising";
  progress: number; partial_findings: number; analysis: Analysis | null;
}
export interface AskResponse {
  status: "answered" | "silent" | "limited" | "off_topic" | "error";
  answer: string | null; citations: Citation[]; kind: Kind | null;
  uncertainty: Uncertainty | null; escalation: string | null; disclaimer: string;
}
export interface Health {
  backend: boolean; llm_local: boolean; model: string;
  offline_mode: boolean; network_guard: string;
}
export interface PrivacyInfo {
  calls: { call: string; when: string; carries_content: boolean }[];
  blocked_events: number;
}
```

11. API client and hooks (starter code)

`src/api/client.ts`

```ts
```

```ts
import type { AskResponse, Health, Job, PrivacyInfo, UploadResult } from "../types";

const BASE = "/api"; // same origin in production; Vite proxy in dev

async function json<T>(res: Response): Promise<T> {
  if (!res.ok) {
    let msg = `Request failed (${res.status})`;
    try { msg = (await res.json()).detail ?? msg; } catch { /* keep default */ }
    throw new Error(typeof msg === "string" ? msg : "Request failed");
  }
  return res.json() as Promise<T>;
}

export async function uploadDocument(file: File): Promise<UploadResult> {
  const body = new FormData();
  body.append("file", file);
  return json<UploadResult>(await fetch(`${BASE}/documents`, { method: "POST", body }));
}
export const startAnalysis = (docId: string) =>
  fetch(`${BASE}/documents/${docId}/analyze`, { method: "POST" }).then((r) => json<{ job_id: string }>(r));
export const getJob = (jobId: string) =>
  fetch(`${BASE}/jobs/${jobId}`).then((r) => json<Job>(r));
export const ask = (docId: string, question: string) =>
  fetch(`${BASE}/documents/${docId}/ask`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ question, history: [] }),
  }).then((r) => json<AskResponse>(r));
export const deleteDocument = (docId: string) =>
  fetch(`${BASE}/documents/${docId}`, { method: "DELETE" });
```

```ts
export const getHealth = () => fetch(`${BASE}/health`).then((r) => json<Health>(r));
export const getPrivacy = () => fetch(`${BASE}/privacy`).then((r) => json<PrivacyInfo>(r));
```

`src/hooks/useAnalysis.ts`

```ts
```

```ts
import { useCallback, useEffect, useRef, useState } from "react";
import { getJob, startAnalysis } from "../api/client";
import type { Analysis, Job } from "../types";

export function useAnalysis() {
  const [job, setJob] = useState<Job | null>(null);
  const [analysis, setAnalysis] = useState<Analysis | null>(null);
  const [error, setError] = useState<string | null>(null);
  const timer = useRef<number | null>(null);

  const stop = () => { if (timer.current) window.clearInterval(timer.current); timer.current = null; };

  const run = useCallback(async (docId: string) => {
    setError(null); setAnalysis(null); setJob(null);
    try {
      const { job_id } = await startAnalysis(docId);
      timer.current = window.setInterval(async () => {
        try {
          const j = await getJob(job_id);
          setJob(j);
          if (j.state === "done") { setAnalysis(j.analysis); stop(); }
          if (j.state === "error") { setError("The analysis could not finish. Check that the local model is running, tl
        } catch {
          setError("Lost contact with the local app. Make sure it is still running."); stop();
        }
      }, 1000);
    } catch (e) {
      setError(e instanceof Error ? e.message : "Could not start the analysis.");
    }
  }, []);
```

```
  useEffect(() => stop, []);
  return { job, analysis, error, run };
}
```

12. Starter component sketches

`FindingCard.tsx`

tsx

```tsx
import type { Finding } from "../types";
import { ConfidenceMeter } from "./ConfidenceMeter";
import { CitationChip } from "./CitationChip";

const LABEL = {
  explicit: "The document says",
  interpretation: "This may mean",
  concern: "Worth checking",
} as const;

export function FindingCard({ finding, onCite }: {
  finding: Finding; onCite: (passageId: string, quote: string) => void;
}) {
  const solid = finding.kind === "explicit";
  return (
    <article
      className={`rounded-[10px] border border-rule bg-surface p-4 border-l-4 ${
        solid ? "border-l-ink" : "border-l-interp border-l-dashed"
      }`}
      aria-label={`${LABEL[finding.kind]}: ${finding.title}`}
    >
      <p className="text-sm font-medium text-muted">{LABEL[finding.kind]}</p>
      <h3 className="font-serif text-lg text-ink">{finding.title}</h3>
      <p className="mt-1 text-ink">{finding.plain_explanation}</p>
      <ul className="mt-3 space-y-2">
        {finding.citations.map((c) => (
          <li key={c.passage_id + c.quote}>
            <CitationChip citation={c} onClick={() => onCite(c.passage_id, c.quote)} />
          </li>
        ))}
```

```
      </ul>
      <div className="mt-3"><ConfidenceMeter uncertainty={finding.uncertainty} /></div>
      {finding.escalate && (
        <p className="mt-3 text-sm font-medium text-escalate">Ask a lawyer about this</p>
      )}
    </article>
  );
}
```

`DocumentViewer.tsx` (highlight core)

```tsx
```

```tsx
import { useEffect, useRef } from "react";
import type { Passage } from "../types";

const norm = (s: string) =>
  s.normalize("NFKC").replace(/[\u2018\u2019]/g, "'").replace(/[\u201C\u201D]/g, '"')
   .replace(/[\u2013\u2014]/g, "-").replace(/\s+/g, " ").toLowerCase();

function split(text: string, quote?: string): { t: string; hit: boolean }[] {
  if (!quote) return [{ t: text, hit: false }];
  // Build a normalised string with an index map back to the original.
  let out = ""; const map: number[] = []; let prevSpace = false;
  for (let i = 0; i < text.length; i++) {
    const ch = norm(text[i]);
    if (/\s/.test(text[i])) { if (prevSpace) continue; prevSpace = true; out += " "; map.push(i); continue; }
    prevSpace = false;
    for (const c of ch) { out += c; map.push(i); }
  }
  const idx = out.indexOf(norm(quote));
  if (idx < 0) return [{ t: text, hit: false }];
  const s = map[idx]; const e = map[idx + norm(quote).length - 1] + 1;
  return [
    { t: text.slice(0, s), hit: false },
    { t: text.slice(s, e), hit: true },
    { t: text.slice(e), hit: false },
  ];
}

export function DocumentViewer({ passages, active, flagged }: {
  passages: Passage[];
  active: { passageId: string; quote: string } | null;
```

```
  flagged: string[];
}) {
  const refs = useRef<Record<string, HTMLElement | null>>({});
  useEffect(() => {
    if (!active) return;
    const el = refs.current[active.passageId];
    el?.scrollIntoView({ block: "center", behavior: "auto" });
    el?.focus();
  }, [active]);

  return (
    <section aria-label="Document" className="font-serif leading-relaxed text-ink">
      {passages.map((p) => {
        const isActive = active?.passageId === p.id;
        const parts = split(p.text, isActive ? active!.quote : undefined);
        return (
          <p
            key={p.id} id={p.id} tabIndex={-1}
            ref={(el) => { refs.current[p.id] = el; }}
            className={`px-4 py-2 outline-none focus-visible:ring-2 ${
              flagged.includes(p.id) ? "border border-dashed border-escalate" : ""
            }`}
          >
            {flagged.includes(p.id) && (
              <span className="mb-1 block font-sans text-xs font-medium text-escalate">
                This text contains instructions aimed at AI tools. They were ignored.
              </span>
            )}
            {parts.map((s, i) =>
              s.hit ? <mark key={i} className="bg-marker text-ink">{s.t}</mark> : <span key={i}>{s.t}</span>
```

```
            )}
          </p>
        );
      })}
    </section>
  );
}
```

13. Setup commands

bash

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm install lucide-react @fontsource/literata @fontsource/public-sans
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

`src/main.tsx` imports fonts locally:

ts

```
import "@fontsource/literata/400.css";
import "@fontsource/literata/600.css";
import "@fontsource/public-sans/400.css";
import "@fontsource/public-sans/600.css";
import "./index.css";
```

`tailwind.config.js` (tokens)

js

```
export default {
  content: ["./index.html", "./src/**/*.{ts,tsx}"],
  theme: {
    extend: {
      colors: {
        paper: "#F4F5F2", surface: "#FFFFFF", ink: "#16212B", muted: "#4A5A68",
        rule: "#D5DAD6", marker: "#CFE8A9", interp: "#6B4FBB",
        attention: "#A15C00", escalate: "#A3262E",
      },
      fontFamily: {
        serif: ["Literata", "Georgia", "serif"],
        sans: ['"Public Sans"', "system-ui", "sans-serif"],
      },
    },
  },
};
```

`vite.config.ts`

ts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
export default defineConfig({
  plugins: [react()],
  server: { proxy: { "/api": "http://127.0.0.1:8000" } },
});
```

Production: `npm run build` outputs `frontend/dist`, which FastAPI serves from `/`. Check the built `index.html` contains no `http://` or `https://` asset URLs.

Note: Tailwind has no `border-l-dashed` utility by default. Use `border-dashed` on the whole border or a small CSS class for the dashed left rule.

14. Build order

Step	Task	Done when
1	Scaffold, Tailwind tokens, bundled fonts, proxy, `types.ts`, `client.ts`	App loads, `/api/health` shows in the header
2	`UploadPanel`, `useDocument`, `DocumentViewer` (plain text)	Real upload shows passages
3	`useAnalysis`, `StageProgress`, `FindingsTab`, `FindingCard`	Real findings appear
4	`CitationChip` + highlight + scroll to passage	Clicking a citation highlights the exact words
5	Says / may mean / worth checking treatments, `ConfidenceMeter`, `EscalationPanel`	Kinds are visibly different without colour
6	`SilentList`, `SummaryTab`, key dates	Silent clauses and summary render
7	`AskTab` with all five statuses	Answered, silent, limited, off-topic, error all render from the real API
8	`PrivacyTab`, `DisclaimerBar`, header badge, delete flow	Declared network calls shown, delete works
9	Error and injection states, a11y pass, mobile tabs	Checklist in section 9 ticked
10	Stretch: export, dark mode	Only if time remains

During development you may use a local fixture of `Analysis` objects so UI work can start before the backend is ready. **Remove the fixture path and verify against the real backend before recording the demo** (gate G3: no mocked or hard-coded outputs).

15. Frontend acceptance checklist

- [ ] Every finding shows a verified quote and "Show in document" works
- [ ] "The document says" and "This may mean / Worth checking" are visually distinct without colour
- [ ] "Not covered in this document" list and the Ask "silent" state both render
- [ ] Legal-verdict question shows the limited state with specific escalation
- [ ] Disclaimer bar always visible; "Local only" badge reflects `/api/health`
- [ ] Privacy tab lists declared network calls
- [ ] Injection test document shows the flagged passage warning
- [ ] Built `index.html` and CSS reference no external URLs; no console errors about blocked requests
- [ ] Works at 360px and desktop widths; keyboard-only use works end to end
- [ ] No hard-coded answers; demo talks to the real backend
- [ ] No secrets and no document content stored in the browser or committed to the repo